

CAREER ROADMAP • 20 STAGES

JAVA FULL STACK DEVELOPER ROADMAP

Beginner → Job Ready

WHAT YOU'LL COVER

Core Java • OOPs • Collections • JDBC
Servlets & JSP • Hibernate • Spring • Spring Boot
MySQL • REST APIs • React.js • Git & GitHub
Projects • Deployment • Interview Preparation

Devika Jangid

Created by

DEVIKA WEB SOLUTIONS

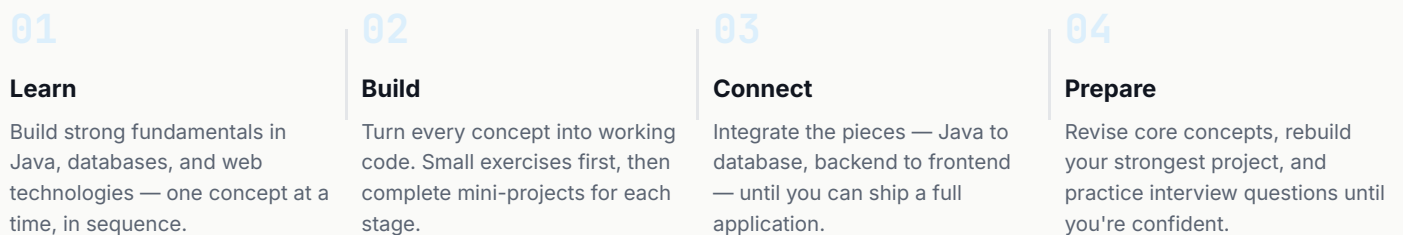
"Learn with direction.
Build with confidence."

INTRO HOW TO USE THIS ROADMAP

Your Roadmap to Becoming a Java Full Stack Developer

Becoming job-ready isn't about learning every technology at once — it's about learning the right things, in the right order, and proving your skill through real projects. Most beginners get overwhelmed because they try to study everything in parallel with no sequence and no checkpoint to measure progress. This roadmap fixes that. It breaks the entire Java Full Stack path into 20 focused stages, each with exactly what to learn, what to practice, and what to build.

The 4-Step Journey



How to use this roadmap

Move through the stages in order — each one is designed to give you exactly what the next stage needs. Don't skip Core Java or SQL to "jump" to Spring Boot; the gaps will show up in interviews and in your projects. Treat each stage as complete only when you've finished the practice tasks and the mini-project, not just read the topic list.

Keep a simple tracker (a notebook or spreadsheet works fine) and check off each topic as you genuinely understand it — not just recognise it. Revisit the Job-Ready Checklist on page 18 every few weeks to see exactly where you stand.

✓ Who this is for

- Students who know basic programming and want a structured path.
- Self-taught learners who've done scattered tutorials but lack direction.
- Anyone preparing for Java Full Stack Developer interviews.

! Before you start

- Basic computer literacy and comfort installing developer tools.
- A working laptop with at least 8GB RAM for IDEs and MySQL.
- 2–4 hours a day of focused, distraction-free study time.

MAP THE FULL PATH, END TO END

Complete Roadmap Overview

Eighteen stages, one continuous path — from your first line of code to a deployed, interview-ready full stack application. Each stage in this book has its own detail page.

- 01 Programming Foundations**
Logic building, variables, loops, functions — the base every language shares.
- 02 Core Java**
Syntax, JVM/JRE/JDK, control flow, arrays, strings, exception handling.
- 03 Object-Oriented Programming**
Classes, encapsulation, inheritance, polymorphism, abstraction.
- 04 Collections & Advanced Java**
List/Set/Map, generics, multithreading basics, Streams & Lambdas.
- 05 SQL & MySQL**
Relational design, queries, joins, normalization, indexing.
- 06 JDBC**
Connecting Java applications directly to a relational database.
- 07 Servlets & JSP**
Classic Java web layer — requests, sessions, and server-rendered views.
- 08 Hibernate / JPA**
Object-relational mapping so Java objects persist without raw SQL.
- 09 Spring Framework**
Inversion of Control and Dependency Injection — the backbone of modern Java backends.
- 10 Spring Boot**
Production-grade REST backends with minimal configuration.
- 11 REST APIs**
Designing clean, stateless HTTP endpoints that any frontend can consume.
- 12 React.js**
Component-based UI, state, hooks, and calling your own APIs.
- 13 Frontend Integration**
Wiring React to Spring Boot — the moment your app becomes "full stack."
- 14 Git & GitHub**
Version control and collaboration — non-negotiable for any developer job.
- 15 Testing**
Verifying your backend and frontend actually do what you think they do.
- 16 Deployment**
Taking a project from your laptop to a live, shareable URL.
- 17 Projects**
Five progressively harder builds that prove your skill on a resume.
- 18 Interview Preparation**
Turning everything you've learned into confident, structured answers.

18

Structured learning stages

4–6

Months, studying consistently

5

Projects built along the way

Programming Foundations

Java is easier to learn once you already think like a programmer. This stage builds that thinking using plain logic — so when Java's syntax shows up next, you're only learning new words for ideas you already understand.

A Core Ideas

- Programming logic & step-by-step problem solving
- Variables and how memory stores values
- Data types — numbers, text, true/false
- Operators — arithmetic, relational, logical

B Control & Structure

- Conditions — if / else if / else / switch
- Loops — for, while, do-while, nested loops
- Functions / methods and reusable logic
- Arrays and Strings as your first data structures

WHAT TO LEARN

- Flowcharts before code
- Reading a problem statement carefully
- Breaking a problem into small steps

WHAT TO PRACTICE

- 20+ small logic problems daily
- Pattern printing with loops
- Array searching & sorting by hand

COMMON MISTAKES

- Jumping to code before planning the logic
- Ignoring edge cases (empty input, zero, negatives)
- Copying solutions instead of writing your own

★ MINI-PROJECT IDEA

Build a simple console-based calculator and a number-guessing game using loops and conditions. Both use only fundamentals, but force you to combine logic, input handling, and control flow — the exact skills Core Java builds on next.

🕒 1-2 WEEKS

Tip — Don't rush past this stage even if you've "seen it before." Weak fundamentals are the #1 reason learners stall out later at OOP and Collections.

Core Java

This is where you actually start writing Java. Understand not just the syntax, but what happens underneath — how Java compiles, runs, and manages your program.

BEGINNER

- Java syntax & program structure
- JVM, JRE, JDK — what each one actually does
- Variables & data types (primitive vs reference)
- Operators & type casting
- Control statements — if, switch, loops
- Methods — parameters, return types, overloading intro

INTERMEDIATE

- Arrays — 1D, 2D, and array of objects
- Strings, StringBuilder & immutability
- Exception handling — try/catch/finally, custom exceptions
- Packages & import structure
- Access modifiers — public, private, protected, default
- static vs instance members

i Why JVM / JRE / JDK matters

The JDK is what you develop with (compiler + tools). The JRE is what runs a Java program (libraries + JVM). The JVM is what actually executes your bytecode on any operating system — this is the reason Java code runs the same way everywhere, which interviewers frequently test.

★ MINI-PROJECT IDEA

Build a console-based Student Marks Analyzer: take marks for multiple subjects, calculate total, percentage and grade, and handle invalid input using exception handling. It touches arrays, methods, strings and exceptions together.

🕒 3-4 WEEKS

Tip — Type out every example yourself instead of copy-pasting. Muscle memory for Java syntax pays off hugely once you reach Spring Boot.

Object-Oriented Programming

OOP is the mindset behind every Java framework you'll learn later — Spring, Hibernate, even React's component model borrows the same thinking. Get genuinely comfortable here; it pays off for the rest of the roadmap.

1 The Four Pillars

- **Encapsulation** — bundling data + behaviour, hiding internals with private fields and getters/setters
- **Inheritance** — a class reusing and extending another with extends
- **Polymorphism** — one method, many forms (overloading & overriding)
- **Abstraction** — exposing what an object does, hiding how

2 Supporting Concepts

- Class & Object — blueprint vs real instance
- Interface — a pure contract of behaviour
- Constructor — default, parameterized, constructor overloading
- this vs super keywords
- Method overloading (compile-time) vs overriding (run-time)

Practical example — Overriding

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
```

Calling `sound()` on a `Dog` reference at runtime executes `Dog`'s version — this is runtime polymorphism, one of the most commonly asked interview concepts.

★ MINI-PROJECT IDEA

Build a Vehicle Rental System: an abstract `Vehicle` class with `Car` and `Bike` subclasses, each overriding a `rentalCost()` method. Practices inheritance, abstraction and polymorphism in one clean model.

 2-3 WEEKS

Tip — Almost every Java interview opens with OOP questions. Be able to explain each pillar with a real example, not just the definition.

Collections & Advanced Java

Real applications constantly store, search, and sort groups of objects. The Collections Framework gives you battle-tested structures for that — and Java 8's Streams & Lambdas let you process them in a clean, modern style.

L List & Queue

- ArrayList — dynamic, index-based
- LinkedList — fast insert/delete
- Iterator for safe traversal

S Set & Map

- HashSet / TreeSet — uniqueness, ordering
- HashMap / TreeMap — key-value pairs
- Comparable vs Comparator for custom sorting

G Generics & Errors

- Generics — type-safe reusable classes/methods
- Checked vs unchecked exceptions
- try-with-resources

MT Multithreading Basics

- Thread vs Runnable
- Thread lifecycle & synchronized blocks
- Why concurrency matters for scalable apps

λ Java 8 Features

- Lambda expressions — concise function syntax
- Stream API — filter, map, collect, reduce
- Functional interfaces & method references

★ MINI-PROJECT IDEA

Build an in-memory Contact Book using a `HashMap<String, Contact>` — add, search, delete, and sort contacts alphabetically using `Comparator`, then rewrite the sorting logic using the Stream API.

🕒 2-3 WEEKS

Tip — Know when to use each collection — ArrayList vs LinkedList, HashMap vs TreeMap — interviewers care more about "why" than "what."

Database & SQL

Every full stack app needs somewhere to store data permanently. This stage focuses on MySQL — relational database design and the SQL you'll write constantly as a backend developer.

D Database Design

- DBMS basics & relational database concepts
- Tables, rows, columns and data types
- Primary key vs Foreign key
- Constraints — NOT NULL, UNIQUE, CHECK, DEFAULT
- Normalization (1NF, 2NF, 3NF) — removing redundancy

Q Querying Data

- SELECT, INSERT, UPDATE, DELETE
- WHERE, ORDER BY, GROUP BY, HAVING
- Joins — INNER, LEFT, RIGHT, self join
- Subqueries & nested SELECTs
- Built-in functions — COUNT, SUM, AVG, DATE functions
- Indexing for faster lookups

SQL Example — Join two related tables

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE d.department_name = 'Engineering';
```

★ MINI-PROJECT IDEA

Design a normalized schema for a Library Management System — Books, Members, and Issued Records — with proper primary/foreign keys, then write 10 practice queries covering joins, grouping and subqueries.

🕒 2 WEEKS

Tip — Practice writing joins from scratch, not just reading them. JDBC and Hibernate both assume you're already fluent in SQL.

JDBC + Servlets + JSP

This stage connects everything you've learned so far into a working web application — Java talking to a database, and Java serving pages over HTTP.

Java



Database



Web Application

J JDBC

- JDBC architecture & driver types
- Connection — establishing a database link
- Statement vs PreparedStatement (SQL injection safety)
- ResultSet — reading query output
- Writing full CRUD operations from Java

S Servlets & JSP

- Servlet lifecycle — init, service, destroy
- HttpServletRequest / HttpServletResponse
- Sessions & cookies for state across requests
- JSP — mixing Java logic into HTML views
- MVC basics — separating logic, data and view

★ MINI-PROJECT IDEA

Build a JDBC-powered Employee Management console app with full CRUD against MySQL, then wrap it in Servlets + JSP so the same operations work through a browser. This is your first real "backend to database to browser" loop.

🕒 2-3 WEEKS

Tip — Always use PreparedStatement over Statement — it's both an interview favourite and a real security practice you'll carry into Spring.

07 NO MORE RAW SQL

Hibernate / JPA

Writing JDBC by hand for every table gets repetitive and error-prone fast. Hibernate lets you work with Java objects directly, and it automatically generates the SQL underneath — this is a distinct, important skill inside the backend journey, not a small add-on to JDBC.

0 ORM Fundamentals

- What Object-Relational Mapping actually solves
- Hibernate architecture — SessionFactory, Session
- Entity classes — @Entity, @Table
- Primary key mapping with @Id
- Column mapping & basic annotations

R Relationships & Queries

- One-to-One relationships
- One-to-Many & Many-to-One mapping
- Many-to-Many with join tables
- CRUD operations through Hibernate/JPA
- JPQL — querying objects instead of raw tables

@ Example — A mapped entity

```
@Entity
class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;
    @ManyToOne
    private Department department;
}
```

★ MINI-PROJECT IDEA

Rebuild your JDBC Employee Management System using Hibernate entities and relationships instead — compare how much boilerplate disappears, and how the Department-Employee relationship becomes a real object link.

🕒 1-2 WEEKS

Tip — Understand what Hibernate is doing behind the scenes — interviewers often ask you to explain the generated SQL for a mapping.

Spring Framework

Spring solves one core problem: objects shouldn't have to create and manage their own dependencies. This single idea — Inversion of Control — is what makes large Java applications maintainable. Keep this stage concise; Spring Boot covers the practical, hands-on side in depth next.

? Why Spring

- Reduces tight coupling between classes
- Makes code easier to test and maintain
- Standard foundation used across the Java industry

IoC Core Concepts

- Inversion of Control (IoC) — Spring manages object creation
- Dependency Injection — constructor & field injection
- Beans — objects managed by the Spring container
- ApplicationContext — the IoC container itself
- Configuration via annotations or XML

M Spring MVC — the shape of it

Spring MVC separates a request into a Controller (handles the request), a Model (the data), and a View (what's returned). Spring Boot builds directly on this pattern to serve JSON instead of web pages — which is exactly what the next stage covers.

🕒 1 WEEK

Tip — Focus on understanding why Dependency Injection matters before memorizing annotations — Spring Boot will make far more sense once this idea clicks.

Spring Boot

Spring Boot is where you'll spend most of your backend career — it strips away Spring's manual configuration and lets you build real REST APIs fast. This is the most important stage in the entire backend path, so it gets the deepest coverage here.

1 Project Setup

- Why Spring Boot over plain Spring
- Project structure & conventions
- Starter dependencies (web, data-jpa, security)
- `application.properties` configuration

2 Building the API

- REST Controllers — `@RestController`
- Service layer for business logic
- Repository layer with Spring Data JPA
- Constructor-based Dependency Injection

3 Production Concerns

- DTOs — separating API shape from entities
- Validation with `@Valid`
- Global exception handling
- MySQL integration via JPA

@ Example — A minimal REST controller

```
@RestController
@RequestMapping("/api/employees")
class EmployeeController {
    private final EmployeeService service;
    EmployeeController(EmployeeService service) { this.service = service; }

    @GetMapping("/{id}")
    EmployeeDTO getById(@PathVariable Long id) {
        return service.findById(id);
    }
}
```

AUTH Authentication Basics

Spring Security fundamentals — filtering requests, protecting endpoints, and hashing passwords before storing them.

JWT JWT Basics

Issuing a signed token on login and validating it on every request — the standard way stateless REST APIs handle authentication.

🕒 4-6 WEEKS

Tip — Build at least two APIs from scratch here — reading tutorials is not the same as debugging your own 500 error at 11pm.

10 THE CONTRACT BETWEEN FRONTEND & BACKEND

REST APIS

REST is the standard way web applications communicate over HTTP. Once your Spring Boot backend speaks REST properly, any frontend — React, mobile, or another service — can consume it the same way.

M HTTP Methods

- GET — retrieve a resource
- POST — create a new resource
- PUT — update/replace a resource
- DELETE — remove a resource

P Request Design

- Path variables — /employees/{id}
- Query parameters — ?page=1&size=10
- Request body in JSON format
- Meaningful, resource-based URL naming

200 Status Codes That Matter

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Server Error

★ MINI-PROJECT IDEA

Build a complete REST API for a Task Manager (CRUD + status filtering by query param), then test every endpoint methodically in Postman — including invalid inputs — before writing a single line of frontend code.

🕒 1-2 WEEKS

Tip — Learn Postman properly — collections, environments, and saved requests will save you hours across every project from here on.

React.js

React is the most in-demand frontend library paired with Java backends. You don't need to master every corner of it — just enough to build clean, working UIs that talk to your Spring Boot APIs.

C Components & Structure

- Components — functional, reusable UI pieces
- JSX — writing markup inside JavaScript
- Props — passing data into a component
- Sensible component folder structure

H State & Hooks

- State — data that changes over time
- Events — handling clicks, input, forms
- `useState` — component-level state
- `useEffect` — side effects & data fetching

R Routing & Forms

- React Router — multi-page navigation
- Controlled form inputs
- Basic client-side validation

API Talking to the Backend

- Axios or fetch for HTTP calls
- Handling loading & error states
- Basic performance — avoiding unnecessary re-renders

★ MINI-PROJECT IDEA

Build a React frontend for the Task Manager API from Stage 10 — list, add, complete, and delete tasks, with loading and error states handled properly using `useState` and `useEffect`.

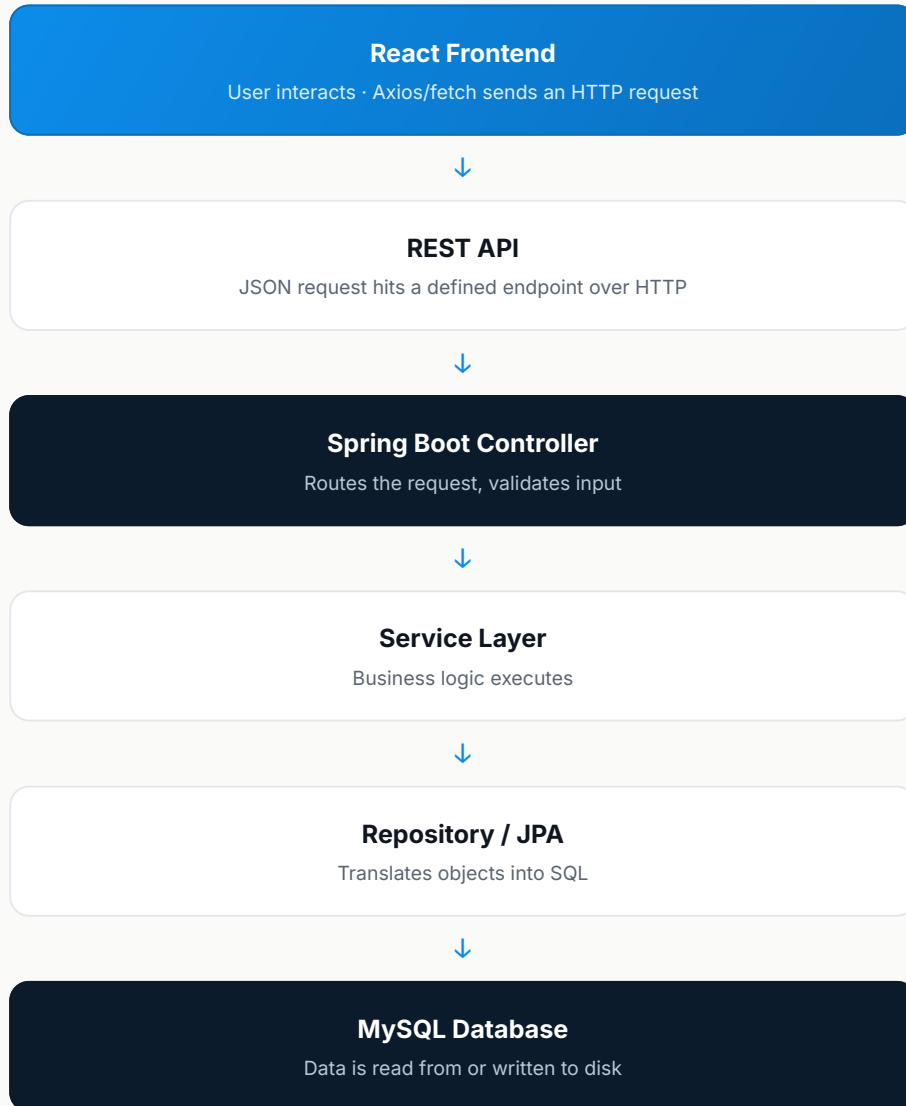
🕒 3-4 WEEKS

Tip — Don't learn React in isolation for too long — connect it to a real API as early as possible so the two skills grow together.

12 WHERE IT ALL CONNECTS

Full Stack Integration

This is the moment your separate skills become one working application. Here's the complete request flow, from a click in the browser to a row in your database — and back.



→ Request goes down

The browser sends JSON → the Controller receives it → the Service applies business rules → the Repository persists it through JPA → MySQL stores the row.

← Response comes back up

MySQL returns data → JPA maps it to Java objects → the Service shapes it → the Controller serializes it to JSON → React updates the UI via state.

🕒 1-2 WEEKS

Tip — Also set up CORS correctly in Spring Boot — it's the single most common blocker beginners hit the first time frontend meets backend.

13 WHAT DEVELOPERS ACTUALLY USE DAILY

Tools & Development Workflow

Knowing Java isn't enough — professional developers work inside a toolchain. Get comfortable with these before your first internship or job so day one isn't a scramble.

Git

VERSION CONTROL

Tracks every change to your code and lets you undo mistakes safely.

GitHub

COLLABORATION

Hosts your repositories and is where recruiters check your project history.

IntelliJ IDEA

JAVA IDE

The industry-standard editor for Spring Boot development.

VS Code

FRONTEND EDITOR

Lightweight, extension-rich editor ideal for React development.

Postman

API TESTING

Send requests to your REST API and inspect responses before frontend integration.

MySQL Workbench

DATABASE GUI

Visually design schemas and run queries without the command line.

Maven

BUILD TOOL

Manages Java dependencies and builds your Spring Boot project.

npm

PACKAGE MANAGER

Installs and manages JavaScript/React dependencies.

Browser DevTools

DEBUGGING

Inspect network requests, console errors, and React component state.

01

Write code
in IDE



02

Test API in
Postman



03

Commit & push
to GitHub



04

Verify in
browser DevTools



05

Deploy &
share the link

🔄 ONGOING

Tip — Commit to GitHub daily, even for small changes — a consistent commit history is one of the first things recruiters glance at.

Project Roadmap

Five projects, beginner to advanced. Each one builds directly on the last — build them in order and your final project becomes a genuine, resume-ready full stack application.

01

CRUD Application

- TECH** Core Java, JDBC, MySQL
- FEATURES** Create, read, update, delete records from a console interface
- TEACHES** Connecting Java to a real database and structuring basic CRUD logic

02

Employee Management System

- TECH** Spring Boot, Hibernate/JPA, MySQL
- FEATURES** Manage employees & departments with relational data and REST endpoints
- TEACHES** Entity relationships, layered architecture, and building a real REST API

03

Online Shopping Application

- TECH** Spring Boot, React, MySQL, REST APIs
- FEATURES** Product catalog, cart, and order flow across frontend and backend
- TEACHES** Your first true full stack integration — React consuming a Spring Boot API

04

Job Portal

- TECH** Spring Boot, Spring Security, JWT, React
- FEATURES** Candidate & recruiter roles, job postings, applications, authentication
- TEACHES** Role-based authentication and multi-user application design

05

Full Stack E-Commerce Application

- TECH** Spring Boot, React, MySQL, JWT, Git, Deployment
- FEATURES** Products, cart, checkout, order history, auth, and a deployed live link
- TEACHES** Tying every stage of this roadmap into one complete, deployed product

15 WHERE DO YOU ACTUALLY STAND?

Job-Ready Checklist

Revisit this page every few weeks. Only tick a box when you could explain that topic out loud, unprompted, in an interview — not just when you've "covered" it.

- | | |
|--|---|
| <p>01 ☒ Core Java
Syntax, JVM, control flow, exceptions</p> <hr/> <p>02 ☒ OOPs
Encapsulation, inheritance, polymorphism, abstraction</p> <hr/> <p>03 ☒ Collections
List, Set, Map, Streams & Lambdas</p> <hr/> <p>04 ☒ SQL
Joins, grouping, normalization</p> <hr/> <p>05 ☒ JDBC
Connections, PreparedStatement, CRUD</p> <hr/> <p>06 ☒ Hibernate
Entities, relationships, JPQL</p> <hr/> <p>07 ☒ Spring
IoC, Dependency Injection, Beans</p> <hr/> | <p>08 ☒ Spring Boot
REST controllers, services, JPA integration</p> <hr/> <p>09 ☒ REST APIs
HTTP methods, status codes, Postman testing</p> <hr/> <p>10 ☒ React
Components, hooks, API integration</p> <hr/> <p>11 ☒ Git/GitHub
Commits, branches, pull requests</p> <hr/> <p>12 ☒ Projects
At least 3 of the 5 roadmap projects, deployed</p> <hr/> <p>13 ☒ Deployment
A live, shareable link to show recruiters</p> <hr/> <p>14 ☒ Interview Preparation
Comfortable answering across every topic above</p> <hr/> |
|--|---|

How to grade yourself honestly

If you hesitate on a topic, don't tick it — go back to that stage's page, redo the mini-project, and revisit the concept until explaining it feels automatic. Job-readiness is a checklist of genuine understanding, not a checklist of pages read.

16 TURNING KNOWLEDGE INTO ANSWERS

Interview Preparation

Sample questions across every topic in this roadmap. Practice answering out loud, then explaining with a real example from your own projects — not just the textbook definition.

CORE JAVA

- Difference between JDK, JRE and JVM?
- Why is Java called platform-independent?
- Checked vs unchecked exceptions?

OOPS

- Overloading vs overriding — with an example?
- Why can't you instantiate an abstract class?
- Composition vs inheritance — when to use which?

COLLECTIONS

- ArrayList vs LinkedList — internal difference?
- How does HashMap actually work internally?
- Comparable vs Comparator?

SQL

- INNER JOIN vs LEFT JOIN, with an example?
- What does normalization solve?
- WHERE vs HAVING — when does each apply?

HIBERNATE

- What problem does ORM actually solve?
- Explain the difference between save() and persist().
- How does lazy loading work?

SPRING

- What is Dependency Injection, in plain terms?
- Constructor injection vs field injection — why prefer one?
- What is a Spring Bean?

SPRING BOOT

- How does Spring Boot reduce configuration vs plain Spring?
- How would you handle a global exception in a REST API?
- What's the role of a DTO, and why not expose entities directly?

REST APIS

- What makes an API "RESTful"?
- PUT vs PATCH — what's the real difference?
- How would you version a public API?

REACT

- What problem do hooks solve vs class components?
- When does useEffect re-run, exactly?
- How do you lift state up between sibling components?

PROJECTS

- Walk me through your project's architecture.
- What was the hardest bug you fixed, and how?
- What would you improve if you rebuilt it today?

🕒 2-3 WEEKS

Tip — Prepare a 2-minute walkthrough of your best project — it's asked in nearly every technical interview, in some form.

Final Action Plan

A realistic pace for someone studying with real focus. Adjust the timeline to your own schedule — but keep the order and the balance between learning, practice, and revision.

DAY 1–30

FOUNDATION PHASE

LEARNING

- Programming Foundations
- Core Java
- OOPs

PRACTICE

- Daily logic problems
- Small Java programs

BUILD

- Calculator & guessing game
- Student Marks Analyzer

DAY 31–60

BACKEND PHASE

LEARNING

- Collections, SQL, JDBC
- Hibernate, Spring, Spring Boot

PRACTICE

- SQL queries daily
- Build 2 REST APIs from scratch

BUILD

- Employee Management System
- Task Manager API

DAY 61–90

FULL STACK PHASE

LEARNING

- React, integration, deployment

REVISION

- Revisit weak topics weekly
- Rebuild your best project cleanly

INTERVIEW PREP

- Daily question practice
- Mock interviews with peers

"You don't need to feel ready to start — you need to start to eventually feel ready. Follow this roadmap one stage at a time, build every project, and trust the process. Job-ready isn't a feeling; it's a checklist you've actually earned."



DEVIKA WEB SOLUTIONS

WEB & PRODUCT STUDIO

"Learn with direction.
Build with confidence."

Created by **Devika Jangid**

This roadmap is designed as a practical learning guide. Technologies and best practices continue to evolve, so keep learning and building.